

Uno. Digital pins can be used both as INPUT and OUTPUT ports. They have two states, i.e., HIGH and LOW. A HIGH state indicates the presence of current in the pin whereas the LOW state indicates the opposite. Digital pins can be used for various purposes such as blinking of LED lights. When set to HIGH, the light will turn on and when set to LOW, it will turn off. This can be done in a loop to create a blinking effect.

Analog pins: There are six analog pins in an Arduino Uno, as shown in Figure 1. They are indexed as A0 to A5 on the board. These pins are used for reading sensor data. A lot of the work done in your Arduino projects will require sensor data. For instance, if you are creating a light that turns on when it detects movement, then that will require you to use a sensor to detect motion. The analog pins can register readings between the values of 0 and 1023.

USB connector: The USB connector helps you to connect your Arduino to the computer so you may upload your code.

Battery power: You can use the battery power connector as shown in Figure 1 to connect your Arduino Uno to a remote battery. This will make your device portable and deployable to places where you may not be able to connect it to a computer or a continuous power source.

The Arduino IDE

To write code for your Arduino project, you will have to download and install the latest Arduino IDE. You can download this from the official Arduino website arduino.cc

Figure 2 shows the default view of an Arduino IDE. Let us go through some of its major features. There are three major components in the IDE, i.e., Toolbar, Status window, and Serial Monitor window.

Toolbar: The Toolbar is the pane on the top as shown in Figure 2. It contains shortcuts to the most commonly used options in the IDE. We shall go through

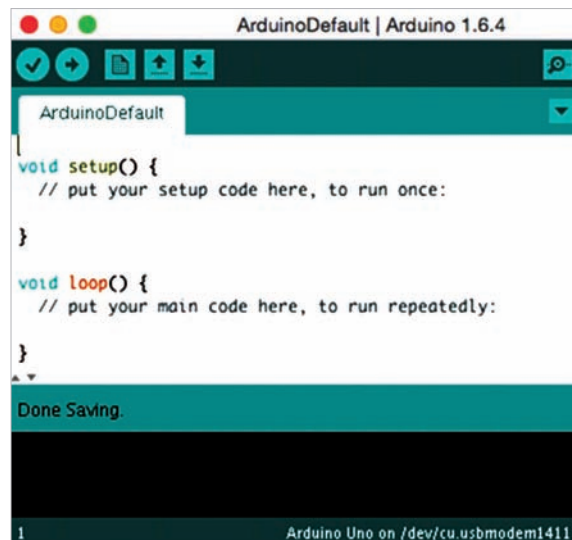


Figure 2: Default view of the Arduino IDE

each shortcut in a left to right order.

1. **Compile:** This is the first button from the left used for compiling and verifying the code for any syntactic or logical errors.
2. **Upload:** The second button from the left, it is used for uploading the code onto the Arduino device. The results of each upload can be viewed in the Status window at the bottom.
3. **New:** This button is used for creating a new code document.
4. **Open:** This button is used for re-opening an already existing code document.
5. **Save:** This is used for saving the progress on a code document.
6. **Serial monitor:** This is the last button on the right that lets you access the Serial Monitor window.

Status window: Status window is the pane shown at the bottom of Figure 2. Whenever you compile or upload your code to the Arduino device, the Status window shows the status of your actions. It is also used for displaying any error messages.

Serial Monitor window: The Serial Monitor window logs all the messages generated by your code. Whenever we use commands such as `Serial.print()` or `Serial.println()` to log any details of our code, they will be displayed on the Serial

Monitor window, as shown in Figure 3.

Arduino programming constructs

Before we begin to write and execute our first Arduino code, let us get familiar with the Arduino programming language. In this section we will cover all the basic constructs used in the Arduino programming language. These are more than enough to

get you started with writing beginner to intermediate level code for your Arduino projects.

The basic programming constructs are listed below.

1. **Int:** Used for integer values such as 12345
2. **Float:** Used for decimal values such as 1.34
3. **char[]:** Used for string values such as 'Car'
4. **HIGH:** Denotes digital pin with current
5. **LOW:** Denotes digital pin without current
6. **INPUT:** To assign a pin to read
7. **OUTPUT:** To assign a pin to set
8. **A0 - A7:** Constants used for each analog pin
9. **0 - 13:** Values used for each digital pin
10. **analogRead():** Returns the value of the analog pin
11. **analogWrite():** Sets the value of the analog pin
12. **digitalRead():** Returns the value of the digital pin
13. **digitalWrite():** Sets the value of the digital pin
14. **Serial.begin():** Initialises the serial monitor
15. **Serial.print():** Outputs a message on the serial monitor
16. **Serial.println():** Outputs a message